

A Systematic Literature Review on Static Application Security Testing (SAST) Tools: Evaluation, Benchmarks, Challenges, and Future Directions

Doaa Dalaq
Information and Computer Science
department, King Fahd University of
Petroleum and Mineral
Dhahran, Saudi Arabia
g202320550@kfupm.edu.sa

Kaniz Fatima Daya
Information and Computer Science
department, King Fahd University of
Petroleum and Minerals
Dhahran, Saudi Arabia
g202319170@kfupm.edu.sa

Alaa Dalaq*
Information and Computer Science
department, King Fahd University of
Petroleum and Minerals
Dhahran, Saudi Arabia
g202320570@kfupm.edu.sa

Muhammed Nazmul Arefin
Information and Computer Science
department, King Fahd University of
Petroleum and Minerals
Dhahran, Saudi Arabia
g202416760@kfupm.edu.sa

Mahmood Khan Niazi
Information and Computer Science
department, King Fahd University of
Petroleum and Minerals
Dhahran, Saudi Arabia
mkniazi@kfupm.edu.sa

Abstract

Context Static Application Security Testing (SAST) tools play an important role in finding software vulnerabilities during the first phase of the software security testing pipeline and development. Yet, the aptness of these tools to find vulnerabilities and their incorporation into Continuous Integration/Continuous Deployment (CI/CD) pipelines is an open research area. Objective: This research provides a systematic analysis and comparison of popular SAST tools (i.e., SonarQube, Checkmarx, and Bandit) based on their capability in vulnerability detection within CI/CD pipelines. Methodology: A systematic literature review (SLR) process was performed, making use of the methodology by Kitchenham and Charters [12]. The review process consisted of six stages: 1) defining research questions, 2) developing a search strategy, 3) selecting relevant studies, 4) assessing study quality, 5) extracting data, and 6) synthesizing the findings. Three performance metrics, including recall, precision, false positive rate, and detection accuracy, were used to evaluate the tool's performance. Results: Results revealed that SonarQube was popular and showed good performance on Java applications, Checkmarx had a higher precision but also generated more false positives, and Bandit was a highly effective tool for Python security flaws. The OWASP Benchmark was most commonly used but lacked diversity due to its real-world, imbalanced nature, whereas the Juliet Test Suite offered more extensive and exhaustive coverage. In terms of reporting evaluation metrics, the True Positive Rate (TPR) and False Positive Rate (FPR) were the most commonly used, whereas the Youden Index (YI), despite being less commonly used, provided a more balanced measure of performance. Conclusion:

The report highlights the necessity of employing a holistic approach to vulnerability detection, demonstrating that no one SAST solution is best at everything. Further studies are to target these tools to improve precision and recall to reduce false positives and develop their combinatorial treatment within the current DevSecOps flow to enhance software security.

CCS Concepts

• Software and Application Security; • Static analysis; • Software Verification and Validation; • Software Vulnerabilities;

Keywords

Static Application Security Testing (SAST), CI/CD, Software Vulnerability Detection, Security Testing Tools, Security Code Review

ACM Reference Format:

Doaa Dalaq, Kaniz Fatima Daya, Alaa Dalaq, Muhammed Nazmul Arefin, and Mahmood Khan Niazi. 2025. A Systematic Literature Review on Static Application Security Testing (SAST) Tools: Evaluation, Benchmarks, Challenges, and Future Directions. In *Proceedings of the 2025 Evaluation and Assessment in Software Engineering Companion (EASE Companion '25)*, June 17–20, 2025, Istanbul, Turkiye. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3727967.3756838>

1 Introduction

Security loopholes and threats are always there to hinder the software developers and businesses around the globe. Security threats have been increasingly active, and software security has received much attention. SAST systems are employed to identify security vulnerabilities in the software's source code before it is run, and are an important component in Secure Software Development Life-cycles (SDLC) today [1]. SAST tools aid developers in detecting vulnerabilities in source code, byte code, or binary code at an early stage of software development, which could help in mitigating security risks pre-deployment [2]. These tools can also be incorporated into Continuous Integration/Continuous Deployment/Continuous

*Corresponding author: Alaa Dalaq, g202320570@kfupm.edu.sa



This work is licensed under a Creative Commons Attribution 4.0 International License. *EASE Companion '25, Istanbul, Turkiye*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1832-8/2025/06
<https://doi.org/10.1145/3727967.3756838>

Delivery CI-CD/CD pipelines to automate security testing and enhance adherence to industry security standards such as OWASP Top [3]. As SAST tools can assess software without running, they become a baseline security control in a DevSecOps world by guaranteeing that gaps are covered between the development-test phase and production readiness [4].

However, as with any tool, SAST tools have challenges that affect their usability. One key problem is disparate detection efficacy—the same SAST tool can produce different results, depending on the type of vulnerability, programming language, or benchmark. Many studies have illustrated that no SAST tool can be considered for complete vulnerability detection for all cases, indicating shortcomings in their efficacy [5]. Also, these kinds of tools tend to produce high false-positive results, namely, these tools label secure code as vulnerable, which can easily cause a loss of confidence in the tool's result [3]. On the other hand, false negatives, which refer to the real vulnerabilities that are not recognized, are more dangerous, since it flag a security vulnerability as false. Besides the above, benchmarking inconsistencies also add to the SAST evaluation issues; various research works based on different data, such as OWASP Benchmark, Juliet Test Suite, and SARD, make comparison of tools difficult [7]. Studies have indicated that SAST tools are effective when applied to synthetic test cases but far less effective on real-world vulnerabilities, as only 12.7% of real-world vulnerabilities are detected by a variety of selected SAST tools in empirical studies [6]. These limitations demonstrate the necessity for a common evaluation strategy upon which SAST tools can be systematically compared based on the same performance metrics.

To cope with these challenges, this Systematic Literature Review (SLR) aims to make a thorough review of SAST tools with an emphasis on their effectiveness for vulnerability detection. Three main research objectives guide this study. Firstly, we evaluate how the different SAST tools perform in detecting defects in software implementations using multiple benchmark programs. Secondly, we benchmark these tools against industry-standard benchmarks such as the OWASP Top 10 to evaluate their real-world effectiveness at vulnerability detection. Thirdly, we determine which SAST tool strikes a good balance between a high recall (discovering vulnerabilities) and low FPs (minimizing unnecessary alerts). Specifically, the following three tools were analyzed, concentrating on SonarQube, Checkmarx, and Bandit, and using the following performance metrics: True Positive Rate (TPR), False Positive Rate (FPR), precision, recall, and F1-score [3], [6]. The results of this research will enable researchers and practitioners to be equipped to choose the right SAST tool for identifying vulnerabilities and lead to more standardised benchmarking in future research work.

This paper is organized as follows: Section 3 describes the approach used for this survey. The results of the review are presented and discussed in Section 3. Implications for research and practice are described in Section 4. Section 5 describes the danger to the validity of the review. Section 6 is about the conclusion and outlook.

2 Related Work

In this section, we have presented an overview of previous surveys and SLRs that were conducted in other contexts on different

aspects of SAST tools and vulnerability detection. Since web applications are at risk from network-based attacks, concerns are raised resulting in the use of a web vulnerability scanner. Lavens et al. [8], performed an extensive study, to measure compliance on 24 commercial vulnerability detectors, with 11 types of vulnerabilities. The performance of several scanners was markedly different, as the results demonstrated. Their study focused on addressing these distinctions by trade-off, taking into consideration the tools available today, especially in the context of discovering broken access control vulnerabilities.

Croft et al. [5] provided empirical evidence on the comparative effectiveness between learning-based SVP models and rule-based SAST tools in security analysis. One of the earlier systematic tests of the two techniques, the study found similar detection rates for SAST tools and SVP models, but stronger performance of SVP models in both Software Vulnerability (SV) detection and assessment. The findings underscore the importance of having scientifically-based views for both researchers and practitioners in terms of selecting and using these tools and implications for future studies in this area.

An empirical research was demonstrated in Aloraini, Alexis, and Palomares. Aloraini et al. [9] conducted a comprehensive study on security warnings reported by SAST tools, particularly analyzing the distribution, half-lives, and decaying rates of these warnings in 116 large C++ projects. The investigation scrutinized industrial software systems, using both commercial and open-source SAST tools and the cregit+, an advanced technology for indexing source code modifications. The results enhance existing work by illustrating the stability properties of security warnings and by emphasizing their role as automated clues to software quality and security exposure, thus lessening the burden of manual inspection. In addition, it also emphasizes a correlation between bug classification and existing fixed vulnerabilities, demonstrating the capacity of SAST tools in discovering security weaknesses in each phase of the software development.

Lipp et al. [10] assessed the effectiveness of SAST tools in detecting vulnerabilities by comparing results for five FOS and one commercial SAST tool across 27 software projects comprising 1.15 million lines of code and 192 confirmed CVEs (ground truth). Their study uses real programs, contrary to most of the previous work, which often uses synthetic datasets to give a more faithful evaluation of the performance of the analyzers. Considering a false negative rate for real vulnerabilities of 47-80%, the findings of the experiment reinforce the limitations of the SAST tools that are applied today. The study also examines the benefits of combining various analyzers, which suggests could improve detection rates by 21 to 34 percentage points at the expense of flagging more functions. This paper gives valuable insights into the challenges of practical vulnerability detection when using static code analysis and paves the way for future work and a better understanding of the tools' limitations.

A comprehensive analysis of seven static analysis tools was presented by Higuera et al. [3] for measuring the efficacy of SAST in identifying vulnerabilities in web applications. The paper is applied on commercial as well as open-source applications that are developed based on J2EE specification and introduces a new benchmarking approach based on the OWASP Top Ten categories

of vulnerabilities. In contrast to previous works that used synthetic datasets, our work focuses on real-world context-based scenarios with several web application vulnerabilities, which makes the evaluation of SAST mechanisms more realistic. The results emphasize the shortcomings of existing SAST methods and also present several areas for improvement in detection prevalence. In addition, the study investigates the trade-off between true and false positives, which provides a practical impact to the security practitioners on how to choose the most effective tools to meet their requirements for web application security scanning tools. As in the results of Lipp et al. [10], their presented work paves the way for future developments that can improve the accuracy and performance of SAST tools, as well as for a better understanding of their limitations in practice.

Vándor et al. [11] conducted an extensive benchmarking in the domain of software security. Their work on early vulnerability detection fosters a comparison of three popular SAST tools systematically (NodeJSScan, ESLint, and CodeQL) with machine learning, line-level vulnerability prediction in JS systems. By analyzing the OpenSSF CVE Benchmark, which offers over two hundred real-world CVEs, the authors show that with this ML-based strategy, 60% of the vulnerabilities in the dataset are located. If ML-based techniques can take a longer time to produce an output with a higher rate of FPs. This strategy has a high capability to identify vulnerabilities that other tools might not detect, especially those related to classes of Common Weakness Enumerations (CWEs). The authors identify potential to better general software security activities in practice, and some strategic directions for development.

Li et al. [6] analyze in-depth SAST tools in an SDLC. The study is based on strict criteria, which ultimately leads to choosing seven SAST tools of free or open-source type from an initial list of 161 candidates. The authors compare and evaluate them on artificial as well as real-world benchmarks like OWASP Benchmark and Java CVE Benchmark. Although SAST tools have good performance on synthetic datasets, in the real world, it is found that only 12.7% of vulnerabilities are detected by the sampling tools we chose. The paper points to issues that are encountered in evaluating these types of SAST tools and emphasizes the need for real-world data sets to obtain meaningful evaluations. The findings provide valuable information to researchers, developers, and practitioners for the development, improvement, assessment, and selection of SAST tools.

Another approach proposes a methodology that leverages the strengths of both real-software and synthetic micro benchmarks and compensates for the limitations of current benchmarks based on real-world software [12]. This process ensures that the test cases are realistic, measurable, and explainable, thereby allowing in-depth evaluation of SAST on real-life scenarios. The researchers considered two SAST tools and generated 10 test cases using their method based on an existing benchmark. Results indicated that the proposed method, however, was substantially sensitive to improvement in performance of identifying vulnerabilities in real-world settings in an intelligible manner.

Pashchenko et al. [7] propose a new method to construct benchmarks for SAST tools automatically called Delta-Bench. The project aims at improving vulnerability realism and ground truth certainty, while also reducing irrelevant findings to mitigate the challenges

of SAST benchmarks. Delta-Bench uses historical vulnerability patches from free and open source software repositories to automatically create benchmarks that separate SAST alerts due to both the vulnerable and patched versions of the software. By flavor selection of dramatic results and noise reduction, the method unites the merits of artificial benchmarks and examination of real-world software. Experimental results on Apache Tomcat show that Delta-Bench can offer an efficient platform to evaluate SAST tools.

Elder et al. [4] presented an empirical investigation to assist project managers and security practitioners in making decisions on choosing software vulnerability detection techniques. They considered an open-source Java-based medical records system, and four types of VMAD methods were considered: Systematic Manual Penetration Testing (SMPT), Exploratory Manual Penetration Testing (EMPT), Dynamic Application Security Testing (DAST), and SAST. The experiment results show that EMPT is more efficient for identifying high-impact vulnerabilities, and EMPT is better at detecting high-impact vulnerabilities, while SAST detects most vulnerabilities in general from the study. The research also shows that manual techniques are as effective as and sometimes more effective than automated techniques when we assess efficiency in terms of Vulnerabilities per Hour (VpH). Each method reveals unique types of vulnerabilities that others might overlook. The results would inform the choice of detection techniques depending on the objectives of the project, the security requirements, and the resource limitations.

Moreover, Li et al. [6] also proposed a study on the effectiveness of query-based SAST (Q-SAST) tools like CodeQL to find vulnerabilities. These tools analyze software databases and encode vulnerability information using SQL-like queries. The evaluation includes 19 common SAST tools, which contain 4 Q-SAST solutions and 3 popular SAST tools with built-in vulnerability detection patterns, and it particularly discusses how effective they are on C/C++ vulnerability detection. The effectiveness and limitations of these tools were tested through synthetic CWE datasets and real-world Common Vulnerabilities and Exposures (CVE) test suites. Moreover, SAST-MT, a metamorphic testing-based software testing framework, is proposed to identify FPs and FNs in CodeQL by examining the defect-fixing Pull Requests (PRs) from three major Q-SAST tools. The study presents essential understanding of the weaknesses, problems, as well as the possible enhancement of Q-SAST tools, which will be valuable for both developers and application users.

3 Method

We conducted an SLR following the process proposed by Kitchenham and Carters [12] to maintain rigour at design, execution, and reporting stages. The review consisted of six stages: searching for research questions, constructing a search strategy, evaluating, selecting studies, assessing quality, and extracting and synthesizing the data.

First, the research questions were developed, and then a structured search strategy was created for identifying the literature. Addin to that, criteria for selecting studies were developed to screen studies and were calibrated with a pilot study. The quality of the studies was checked with the aid of a quality assessment checklist.

Data extraction was done using a structured form (piloted), and the synthesis method was independent of research objectives.

To maximize replicability and minimize bias, the review protocol was prepared through extensive group discussion. More details about each stage are given in the following subsections.

3.1 Research Questions

The objective of this SLR is to assess SAST tools for detecting software vulnerabilities with the help of the OWASP benchmark. Since early vulnerability detection is crucial in the software development lifecycle, we evaluate the effect of SAST tools on a multi-language and repository perspective.

The research focuses on three key questions:

1. Which SAST tool is the most effective in detecting vulnerabilities in open-source code?
2. How do different SAST tools perform in vulnerability analysis using the OWASP benchmark?
3. What evaluation metrics are commonly used to assess SAST tool performance?

3.2 Search Strategy

A planned search strategy was used to search for studies regarding SAST tools and their efficacy on detecting vulnerability. The search was performed in six electronic databases: Google Scholar, ACM Digital Library, ScienceDirect, IEEE Xplore, SpringerLink, and Scopus. The search terms were based on the research questions and it was structured to include synonyms for each term and connectors using the Boolean operators (AND/OR).

3.3 Study Selection Process

1. **Initial Search:** Retrieved **1186 candidate papers**.
2. **Filtering by Relevance:** Narrowed to **73 studies** based on inclusion criteria.
3. **Snowballing Technique:** Manually reviewed reference lists, finalizing **26 studies**.

3.4 Quality Assessment

Each study was **scored (0–4 points)** using the following criteria:

1. **Clarity, coherence, and accuracy of results**
2. **Alignment with the proposed research questions**
3. **Acknowledge any limitations in our study**
4. **Clearly defined research aims**

Quality Results:

- **High Quality (3.5 – 4.0):** 15 studies (included)
- **Medium Quality (2.5 – 3.0):** 11 studies (included)
- **Low Quality (<2.5):** 47 studies (excluded)

3.5 Justification of SLR over MLR

In this study, an SLR approach rather than MLR was adopted because the former was more appropriate for a detailed, qualitative synthesis of existing empirical evidence. Although MLRs allow for broad coverage of trends in the research, SLRs allow for deeper, more critical analyses and structured comparison of studies to be made due to strict inclusion criteria. To measure the effectiveness, precision, and recall of individual SAST tools over known

benchmarks, a systematic amalgamation was required that could aggregate and interpret the finer nuances, thereby guiding future research. Accordingly, SLR was considered more suitable to provide the necessary depth and focus of the study.

4 Results and Discussion

The following section covers the full process from initial database searching of articles to final decisions of incorporating studies within this review. This review also follows the snowballing approach to widen the scope of the study in addition to qualitative judgements, aided by a strict selection criterion.

4.1 Overview of Selected Studies

A literature review was performed in six main digital databases including Google Scholar, ACM Digital Library, Science Direct, SpringerLink, and Scopus. The first stage was to develop a comprehensive bibliography of publications on the research topic. The search was designed to be comprehensive only including the research within the scope of our study. Hence, we found 73 papers relevant to our research question. Diverse perspectives on the topic were subject to further examinations in this category of articles.

4.2 Applying Selection Criteria and Quality Assessment

Following the first round of paper selection, a strict procedure was adopted. The inclusion criteria of our research, according to our methodology, were related to relevance to the research question, publication date, citation count, and journal/conference awards. A quality assessment checklist was also applied to each paper, questioning about research methodology, clarity of results and importance of findings. This rigorous process allowed for the inclusion of only studies with high relevance and quality. We were thus able to focus our research on the most important and relevant literature in our domain of study by refining the original set of 73 papers to 26 through this selective filtering.

4.2.1 Study of Snowballing. In addition to the selected databases, we also used the snowball approach as an additional strategy to identify other related articles. This included the list of references for the 26 papers that were found in the step above, and took us to additional studies that were not found when the database was searched. This approach was also used to verify the completeness of our search strategy.

Using snowballing, we found an additional 6 papers, resulting in an increase from 67 to 73 in the number of studies included. Such papers were added to our dataset to make it as sound and complete a groundwork for further analysis as possible.

4.3 Types of SAST Tools (RQ1)

We observed 57 SAST tools that can be used to detect vulnerabilities from the identified literature. The usage percentage of SAST tools in the chosen studies was demonstrated and measured. From the percentage provided, of the available studies, FindSecBugs was the most implemented.

Many studies have shown the usefulness of various SAST tools in uncovering security vulnerabilities. With an approach and benchmark we developed for this study only recently, the study evaluated seven static analysis tools. The research defined the most efficient technologies per category, considering relevant performance indices. The study focused on QSAST and SAST tools (particularly, Q-SAST), as well as presented significant novel information regarding SAST tools. A comprehensive comparison of SAST tools, including CodeChecker (CCH), CodeQL (CQL), CommSCA (CSA), FlawFinder (FLF), Cppcheck (CPC), and Infer (IFR), was made considering benchmark selection and tool-specific evaluation metrics. This comprehensive study contributes to our better comprehension of the pros and cons of a set of SAST tools and presents recommended advice to choose one SAST tool for SDP.

4.4 Benchmark Used for Vulnerability Evaluation (RQ2)

According to the second research question (RQ2), specific benchmarks, including OWASP, Juliet, Maven, as well as CWE, have been investigated. The focus of the study is the fine-grained comparison of accuracy when performing vulnerability analysis. The efficacy of these evaluations depends on the details of the project data and the methods for validation.

A detailed overview of the benchmarks used in this research, including their goals, descriptions, and relevance to SAST tools, is presented. These descriptions can give a special view of SV discovery by showing the characteristics and purpose of each benchmark. This kind of benchmark serves different goals: from analyzing vulnerabilities related to hosts and dependencies, feedback for the effectiveness of SAST tools on different codebases.

The above analysis includes the well-known OWASP Top Ten project benchmark. This baseline can be used by end users to compare tools, such as Fortify, as long as the user adapts the tool to fit more modern OWASP Top Ten projects. Although SAST tools perform well on synthetic benchmarks like the OWASP Benchmark, their detection rate on real-world vulnerabilities from the Java CVE Benchmark is significantly worse. The OWASP reference benchmark is a foundation to be used as a reference to compare any SAST, DAST, and IAST tools and how well each SAST tool discovers specific vulnerabilities, and likewise for DAST and IAST tools. We use the OWASP ASVS and the OWASP Top Ten as benchmarks for evaluating SAST. The Juliet Test Suite was also used for research purposes to overcome the shortcomings of OWASP. A benchmark is used, which comprises a dataset of 27 F/OSS projects and 192 known security bugs.

4.5 Metrics for Evaluating Performance (RQ3)

Several key evaluation metrics were identified for assessing SAST tools. These include:

- **True Positive (TP):** Correctly identifying security flaws in the code.
- **False Positive (FP):** Incorrectly identifying non-vulnerabilities as vulnerabilities.
- **True Negative (TN):** Correctly determining that a piece of code contains no vulnerabilities.

- **False Negative (FN):** Failing to identify actual vulnerabilities.
- **Recall:** The tool's capacity to identify all genuine vulnerabilities in the codebase.
- **Precision:** The accuracy of the tool's favorable forecasts.
- **F-measure:** A balance between precision and recall.
- **F1-score:** A measure of the model's accuracy.
- **Accuracy:** The overall effectiveness of the tool.
- **Youden Index:** A combined assessment of recall and specificity.

The results of the study support RQ3, which considers evaluation metrics – that addresses the assessment of vulnerability analysis in software security, and we identified a thorough analysis of multiple evaluation metrics concerning the assessment of SAST tools.

Recall, Precision, and F1-score were used as the evaluation criteria for the assessment of the OWASP Benchmark. The Java CVE Benchmark then measures the level of detection and presence of CVEs automatically. The TPR, FPR, and the YI are adopted to measure the performance of the vulnerability scanner in the paper. Kuszczynski and Walkowski [2] computes FP, FN, and analyzes failures for SAST tools. Other evaluation metrics (i.e., precision and sensitivity) are used to report the comparison results of their study.

In general, although SAST tools can help identify vulnerabilities, their effectiveness depends on the benchmark chosen. For our future work, we will concentrate on updating assessment frameworks and developing more powerful security testing tools to enhance the accuracy of vulnerability detection.

5 Threats to Validity

We identified three aspects as potential threats concerning the validity of the present review methodology: bias in the selection of trials, publication bias, and the possibility of errors in data extraction. These can influence the generalizability and reliability of our results.

5.1 Study Selection Bias

Search strategies and inclusion criteria can create bias in the selection of studies. Although our search terms were similar and developed to match our research questions and applied to different electronic databases, it still cannot be free of risk since some eligible studies may not contain these terms in their titles, abstracts, or keywords, and thus were missed by automated searches.

To minimize this risk, we complemented our search by searching Google Scholar and by applying a snowballing approach to the reference lists of the included studies. In addition, selection criteria were developed to ensure that these studies served those objectives and final selection was the result of independent reviews from multiple reviewers with disagreements resolved by all members of the team. However, despite these approaches, there may be a few key studies that have been missed.

5.2 Publication Bias

Publication bias is another potential concern, that is, positive results of SAST tools are more likely to be published than negative or inconclusive results. This could result in an over-estimation of the

Table 1: Comparing the features and performance of SonarQube, Checkmarx, and Bandit.

Feature	SonarQube	Checkmarx	Bandit
Language Support	Java, C#, JavaScript, more	20+ languages (Java, C#, JavaScript)	Python only
Integration	Easy CI/CD Integration	Strong CI/CD and SCM integration	Simple integration via CLI
Customization	High (custom rules, plugins)	High (custom queries)	Moderate (configurable severity levels)
Strengths	Broad coverage, great reporting	Deep scanning, high customizability	Lightweight, Python-specific
Weaknesses	Sometimes misses complex vulnerabilities	High FP overhead	Limited language support
License	Open-source + Commercial Editions	Commercial	Open-source

efficacy of these tools, since the reports of poor performance or restriction of SAST tools may not be accessible.

To overcome this limitation, we have looked for comparative studies that objectively evaluate the performance of different SAST tools with a fair view about their efficacy: such studies were among our inclusion criteria. Nevertheless, the addition of gray literature, including technical reports, unpublished studies, and thesis work, which may contain relevant but non-peer-reviewed data, would also be necessary to avoid potential bias.

5.3 Data Extraction Errors

Data extraction error is a major threat to validity in systematic reviews. Misunderstandings about how to extract results from included studies could result in inaccurate conclusions regarding the provision of SAST tools. These mistakes are likely to originate from misinterpretations of evaluation metrics, measurement conditions, and methodologies, or simple human errors during data crawling. To overcome this, several measures were applied:

The Standardized form of data extraction must be used for consistency. Data from each study were independently abstracted by several reviewers. Any discrepancies related to collected data were resolved by consensus discussion. Although the above-mentioned precautions are taken in the studies, it is not possible to avoid human errors entirely, and some minor errors in the extracted information might still exist.

5.4 Generalizability of Results

Our findings were compromised by the limitations of the included studies. Most examined studies were performed using certain SAST tools, datasets, and evaluation benchmarks, which leads to results with less possibility of generalization to other security testing contexts. Furthermore, some programming languages and software development environments could be underrepresented in our dataset.

To improve generalizability in future studies, additional research should consider:

1. A wider range of programming languages beyond the ones commonly tested.
2. Real-world applications in addition to synthetic benchmarks.

3. Integration with other security testing techniques such as DAST and IAST to assess hybrid approaches.

5.5 Potential Bias in Benchmark Selection

The criteria by which SAST tools are evaluated may also be biased. Some benchmarks, like OWASP, Juliet, and CWE, concentrate on pre-defined security bugs that might not be comprehensive in terms of real-world threats. Furthermore, some SAST tools could have been tuned for these benchmarks, and therefore, we may have the perception of top performance on controlled experiments but not on real cases.

The new approach should be evaluated on a more diverse set of benchmarks. Incorporation of real-world examples, rather than just benchmarks. Explore how machine learning vulnerability detection stacks up to rule-based SAST tools of old.

Despite the robust approach adopted by this study in reviewing with minimal bias and presenting an extensive review, we acknowledge that limitations introduced through the systematic review of literature are inherent. Our methods also sought to minimise these threats: we expanded our search strategies, extracted the data independently, and used more than one measure of quality. Yet, future studies should try to further limit selection bias, increase benchmark diversity, and include more realistic software applications to improve the reliability and generalizability of SAST tool evaluations.

6 Conclusion

This SLR provided a comprehensive overview of the current state of research on SAST tools, focusing on their alignment with various benchmarks, such as the OWASP Top 10 security threats, Juliet Test Suite, Maven, CWE, and others. By synthesizing studies from multiple scholarly sources, we gained valuable insights into the effectiveness, challenges, and future directions of SAST tools in addressing software security vulnerabilities.

Acknowledgments

The authors acknowledge the use of OpenAI's ChatGPT for limited support in proofreading and rephrasing portions of this paper. No generative AI tool was used to generate original scientific content,

data, or results. All substantive content and analysis presented in the paper are the original work of the authors.

References

- [1] "IEEE Xplore Full-Text PDF:" Accessed: Feb. 28, 2025. [Online]. Available: [https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=\\$&arnumber=\\$10833895&tag=\\$1](https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=$&arnumber=$10833895&tag=$1)
- [2] [2] K. Kuszczynski, and M. Walkowski, "Comparative Analysis of Open-Source Tools for Conducting Static Code Analysis," *Sensors* 2023, Vol. 23, Page 7978, vol. 23, no. 18, p. 7978, Sep. 2023, doi: 10.3390/S23187978.
- [3] [3] J. R. B. Higuera, J. Bermejo, J. A. S. Montalvo, J. C. Villalba, and J. J. N. Pérez, "Benchmarking Approach to Compare Web Applications StaticAnalysis Tools Detecting OWASP Top Ten Security Vulnerabilities," *Computers, Materials & Continua*, vol. 64, no. 3, pp. 1555–1577, Jun. 2020, doi: 10.32604/CMC.2020.010885.
- [4] [4] S. Elder *et al.*, "Do I really need all this work to find vulnerabilities?: An empirical case study comparing vulnerability detection techniques on a Java application," *Empir Softw Eng*, vol. 27, no. 6, pp. 1–78, Nov. 2022, doi: 10.1007/S10664-022-10179-6/TABLES/16.
- [5] [5] R. Croft, D. Newlands, Z. Chen, and A. M. Babar, "An empirical study of rule-based and learning-based approaches for static application security testing," in *International Symposium on Empirical Software Engineering and Measurement*, IEEE Computer Society, Oct. 2021. doi: 10.1145/3475716.3475781.
- [6] [6] Z. Li, Z. Liu, W. K. Wong, P. Ma, and S. Wang, "Evaluating C/C++ Vulnerability Detectability of Query-Based Static Application Security Testing Tools," *IEEE Trans Dependable Secure Comput*, 2024, doi: 10.1109/TDSC.2024.3354789.
- [7] [7] I. Pashchenko, S. Dashevskiy, and F. Massacci, "Delta-Bench: Differential Benchmark for Static Analysis Security Testing Tools," *International Symposium on Empirical Software Engineering and Measurement*, vol. 2017–November, pp. 163–168, Dec. 2017, doi: 10.1109/ESEM.2017.24.
- [8] [8] E. Lavens, P. Philippaerts, and W. Joosen, "A Quantitative Assessment of the Detection Performance of Web Vulnerability Scanners," in *ACM International Conference Proceeding Series*, Association for Computing Machinery, Aug. 2022. doi: 10.1145/3538969.3544416.
- [9] [9] B. Aloraini, M. Nagappan, D. M. German, S. Hayashi, and Y. Higo, "An empirical study of security warnings from static application security testing tools," *Journal of Systems and Software*, vol. 158, p. 110427, Dec. 2019, doi: 10.1016/J.JSS.2019.110427.
- [10] [10] S. Lipp, S. Banescu, and A. Pretschner, "An Empirical Study on the Effectiveness of Static C Code Analyzers for Vulnerability Detection; An Empirical Study on the Effectiveness of Static C Code Analyzers for Vulnerability Detection," vol. 12, doi: 10.1145/3533767.
- [11] [11] N. Vándor, B. Mosolygó, and P. Hegedű, "Comparing ML-Based Predictions and Static Analyzer Tools for Vulnerability Detection", Accessed: Feb. 28, 2025. [Online]. Available: <https://cwe.mitre.org>
- [12] [12] B. Kitchenham, "Guidelines for performing Systematic Literature Reviews in Software Engineering," 2007. [Online]. Available: <https://www.researchgate.net/publication/302924724>